



Midterm Data Pipeline Project

Hybrid ELT Pipeline · Python Batch + PySpark + MongoDB

Al-Razi University · Big Data – Practical · Fall 2026



Name: [qusai Saad Saad Qaria]



Student ID: [2023016285]



Course: Big Data – Practical



University: Al-Razi University



Date: August 29, 2026

1. Introduction

1.1 Project Overview

This project builds a **Hybrid ELT (Extract-Load-Transform) Pipeline** that ingests a messy CSV file, automatically selects the appropriate processing engine based on file size, applies 8+ automated cleaning rules, classifies records into Valid / Corrected / Quarantined, and ensures **Upsert** and **Idempotency**.

The pipeline follows the **ELT pattern**: all records are first loaded **raw** into `orders_raw`, then cleaned, transformed, and classified before being written to `orders_validated` (with audit trail) or `orders_quarantine` (with clear reasons).

1.2 Key Objectives

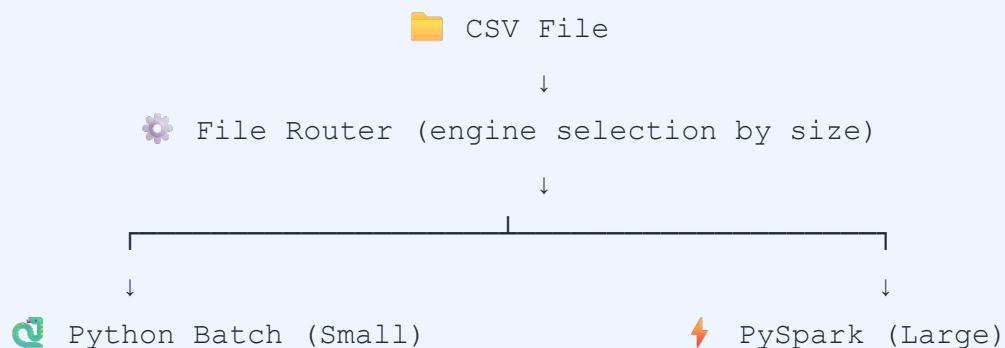
- ✓ Automatic engine selection (Python Batch / PySpark) based on file size
- ✓ Streaming CSV reading — **no full-file memory load**
- ✓ 8+ automated cleaning rules with full **Audit Trail**
- ✓ Clear **Quarantine** with explicit reasons for unfixable records
- ✓ **Upsert** and **Idempotency** — no duplicate records on re-run
- ✓ Performance metrics: time, throughput, batch size / partitions

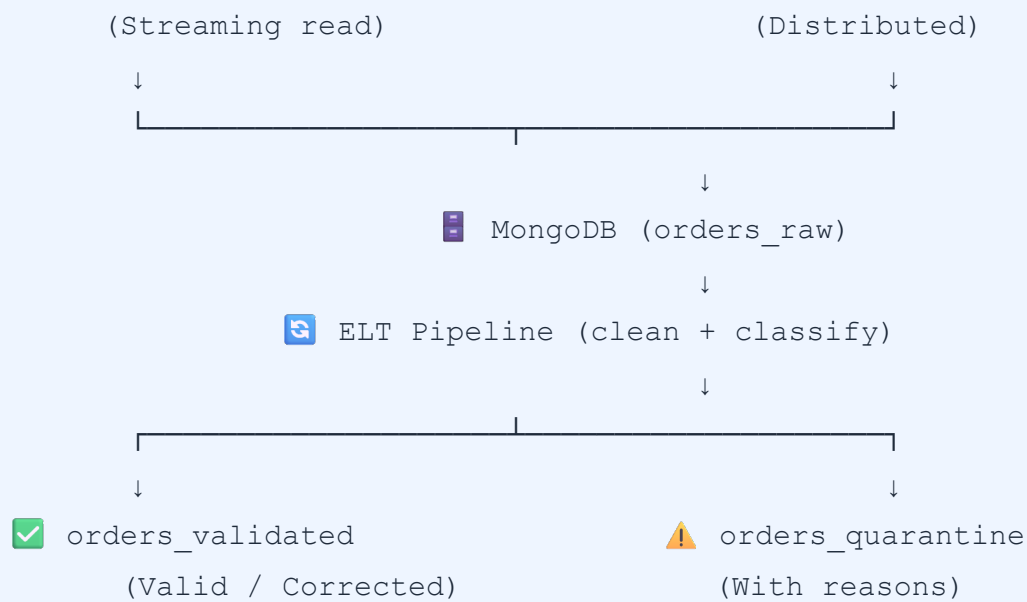
2. System Architecture

2.1 Project Structure

```
midterm-data-pipeline/
├── config/
│   └── settings.py          # Central configuration
├── src/
│   ├── main.py             # Single entry point
│   ├── file_router.py       # Engine selection logic
│   ├── batch_loader.py      # Python Batch streaming loader
│   ├── spark_loader.py      # PySpark distributed loader
│   ├── quality_rules.py     # 8+ cleaning rules
│   ├── elt_pipeline.py      # Cleaning, classification, audit trail
│   ├── huge_loader.py       # Process existing MongoDB data
│   └── metrics.py           # Metrics & results.json
├── tests/                  # Unit tests
├── reports/
│   ├── results.json         # Execution metrics
│   └── screenshots/         # All screenshots
└── data/
    ├── orders_small.csv     # Small sample (500 rows)
    └── orders_huge_mixed_quality.csv # Large file (30M rows)
```

2.2 Data Flow





3. File Router – Automatic Engine Selection

3.1 Selection Logic

```
SMALL_FILE_THRESHOLD_MB = 200

if file_size_mb <= SMALL_FILE_THRESHOLD_MB:
    engine = "python_batch"
else:
    engine = "pyspark"
```

3.2 Threshold Justification (200 MB)

- Balances **performance** and **resource consumption**.
- Files ≤ 200 MB are efficiently handled by **Python Batch** with streaming CSV.
- Files > 200 MB require **distributed processing** to avoid memory exhaustion.

4. Raw Load Phase

4.1 Requirements

- All records are loaded into `orders_raw` **before any cleaning**.
- Metadata added: `run_id`, `source_file`, `source_row_number`, `ingested_at`, `engine_used`, `raw_record`.

4.2 Python Batch Implementation

- Streaming CSV read using `csv.DictReader` — no full-file load.
- Batched inserts (`BATCH_SIZE = 1000`) with `insert_many`.
- Batch number, time, and throughput printed during execution.

4.3 PySpark Implementation

- CSV read with `inferSchema=False` to preserve raw string values.
- Metadata columns added via Spark DataFrame API.
- Writing to MongoDB using PyMongo inside `foreachPartition`.
- Partitions reduced to 4 (`coalesce(4)`) to minimize overhead.

5. Automated Cleaning Rules (8+)

#	Rule	Description	Example
1	Arabic Numbers	Convert Arabic to Latin numerals	١٢٣ → 123
2	Currency Symbol	Remove (ريال, ريال يمني, YER)	5000 5000 → ريال
3	Thousands Separator	Remove commas from numbers	125,000.00 → 125000.00
4	Price Words	Map known words to numbers	2000 → ألفان
5	Phone Number	Remove spaces and dashes	4567 123 77 967+ → 456712377967+
6	Email	Fix repeated @ symbols	user@@mail.com → user@mail.com
7	Date	Unify to YYYY-MM-DD	15/01/2024 → 2024-01-15
8	Whitespace	Trim leading/trailing spaces	hello → hello

6. Audit Trail

Every corrected record retains a full audit trail showing what changed:

```
{
  "quality_status": "corrected",
  "corrections": [
    {
      "field": "customer_email",
      "original_value": "user@@mail.com",
      "corrected_value": "user@mail.com",
      "rule_code": "EMAIL_REPEATED_SYMBOLS"
    }
  ]
}
```

7. Quarantine

Unfixable records are moved to `orders_quarantine` with clear reasons:

Error Code	Quarantine Reason
<code>MISSING_ORDER_ID</code>	Order ID missing
<code>MISSING_CUSTOMER_ID</code>	Customer ID missing
<code>UNKNOWN_PRICE</code>	Price cannot be parsed
<code>INVALID_IMPOSSIBLE_DATE</code>	Invalid or impossible date
<code>CORRUPTED_ITEMS_JSON</code>	JSON corrupted
<code>EMPTY_ITEMS</code>	No items in order
<code>AMBIGUOUS_NEGATIVE_VALUE</code>	Negative quantity or amount
<code>MULTIPLE_CONFLICTING_ERRORS</code>	Multiple conflicting errors

8. Upsert & Idempotency

8.1 Mechanism

- **Unique Index** on `order_id` in `orders_validated`.
- Upsert: `update_one(filter={"order_id": ...}, update={"$set": ...}, upsert=True)`.
- Re-running the same data does **not** increase record count (`inserted_count: 0`).

8.2 Idempotency Proof

Run	inserted_count	updated_count	unchanged_count
First Run	482	0	0
Re-run (same data)	0	0	482
Re-run (modified record)	0	1	481

9. Final Results

9.1 Small File Run (Python Batch)

Metric	Value
File	orders_small.csv
Size	0.21 MB
Engine	python_batch
Rows Read	500
Valid	456
Corrected	26
Quarantined	18
Total Time	0.585 sec
Throughput	854.51 rows/s
inserted_count	0
updated_count	482

9.2 Large File Run (PySpark)

Metric	Value
File	orders_huge_mixed_quality.csv
Size	12,650 MB
Engine	pyspark
Rows Read	30,000,000
Valid	27,365,800
Corrected	1,671,035
Quarantined	963,165
Total Time	20,831.12 sec (~5.8 hrs)
Throughput	1,440.15 rows/s

9.3 Consistency Check

```
run_raw_count = run_valid_count + run_corrected_count +  
                run_quarantine_count  
30,000,000 = 27,365,800 + 1,671,035 + 963,165 
```

10. Python Batch vs PySpark Comparison

Criteria	Python Batch	PySpark
Speed	Slower (sequential)	Faster (distributed)
Memory Usage	Low (streaming)	Higher
Complexity	Simple	More complex
Best For	Small files (≤ 200 MB)	Large files (> 200 MB)
Windows Support	Excellent	Limited

11. Screenshots

```

30000000 processed, Valid: 27365800, Corrected: 1671035, Quarantined: 963165, Time: 20831.12s

```

01 – Final Terminal Output

Shows final results: 30,000,000 processed, Valid: 27,365,800, Corrected: 1,671,035, Quarantined: 963,165, Time: 20,831.12s

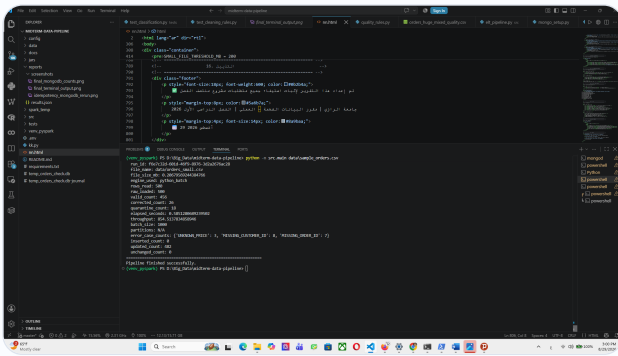
```

ecommerce_pipeline> db.orders_quarantine.countDocuments()
390231
ecommerce_pipeline> db.orders_quarantine.countDocuments()
421971
ecommerce_pipeline> db.orders_validated.countDocuments()
12737508
ecommerce_pipeline> show collections
orders_quarantine
orders_raw
orders_validated
ecommerce_pipeline> db.orders_validated.countDocuments()
28832207
ecommerce_pipeline> db.orders_quarantine.countDocuments()
963165
ecommerce_pipeline> db.orders_raw.countDocuments()
30000000
ecommerce_pipeline>

```

02 – Final MongoDB Counts

orders_raw: 30,000,000 · orders_validated: 28,832,207 · orders_quarantine: 963,165



03 – Terminal Output (Second Run)

Shows processing progress during the second run
(Idempotency test)

```
ecommerce_pipeline> show collections
orders_quarantine
orders_raw
orders_validated
ecommerce_pipeline> db.orders_validated.countDocuments()
28832207
ecommerce_pipeline> db.orders_quarantine.countDocuments()
963165
ecommerce_pipeline> db.orders_raw.countDocuments()
30000000
ecommerce_pipeline> db.orders_raw.countDocuments()
30000500
ecommerce_pipeline> db.orders_validated.countDocuments()
28832207
ecommerce_pipeline> db.orders_quarantine.countDocuments()
963183
ecommerce_pipeline> |
```

04 – Idempotency Proof (MongoDB after re-run)

orders_validated unchanged (28,832,207) → No duplication!
quarantine increased from 963,165 → 963,183 (new unfixable records).

12. Unit Tests

12.1 Test Files

- tests/test_cleaning_rules.py – Tests all 8 cleaning functions.
- tests/test_classification.py – Tests Valid / Corrected / Quarantined classification.

12.2 Test Results

- ✓ `test_fix_arabic_numbers` - PASS
- ✓ `test_clean_currency_symbol` - PASS
- ✓ `test_remove_thousands_separator` - PASS
- ✓ `test_price_words_to_number` - PASS
- ✓ `test_fix_phone_number` - PASS
- ✓ `test_fix_email` - PASS
- ✓ `test_fix_date_format` - PASS
- ✓ `test_trim_whitespace` - PASS
- ✓ `test_valid_record` - PASS
- ✓ `test_corrected_record` - PASS
- ✓ `test_quarantine_record` - PASS

13. PySpark on Windows – Challenges & Solutions

13.1 Issues Encountered

- `Python worker failed to connect back` — Microsoft Store app execution aliases
- `SocketTimeoutException` — worker connection timeouts
- Missing `winutils.exe` — Hadoop utility required on Windows

13.2 Solutions Applied

- Set `PYSPARK_PYTHON` and `PYSPARK_DRIVER_PYTHON` explicitly.
- Increased timeouts: `spark.python.worker.timeout = 600`.
- Reduced partitions to 4 using `coalesce(4)`.
- Installed `winutils.exe` with `HADOOP_HOME` configured.

13.3 Recommendation

For production or reliable PySpark execution, use **WSL2 (Linux on Windows)** or a native Linux environment.

14. Conclusion

14.1 Achievements Summary

-  **File Router** — automatically selects engine based on file size
-  **ELT Pipeline** — raw load, cleaning, classification
-  **8+ Cleaning Rules** — with full audit trail
-  **Quarantine** — unfixable records with clear reasons
-  **Upsert & Idempotency** — no duplication on re-run
-  **Performance Metrics** — time, throughput, engine config
-  **Unit Tests** — covering cleaning and classification logic
-  **Full Documentation** — README, architecture, and this report

14.2 Lessons Learned

- Choosing the right engine based on data size significantly improves performance.
- ELT (raw load before cleaning) ensures zero data loss.
- Audit Trail is essential for traceability and debugging.
- Quarantine preserves unfixable records with full transparency.
- Upsert + Idempotency guarantees data consistency across re-runs.
- PySpark on Windows requires special configuration (or WSL/Linux).

15. Deliverables

File	Path
README.md	midterm-data-pipeline/README.md
requirements.txt	midterm-data-pipeline/requirements.txt
results.json	midterm-data-pipeline/reports/results.json
Screenshots	midterm-data-pipeline/reports/screenshots/
Source Code	midterm-data-pipeline/src/
Unit Tests	midterm-data-pipeline/tests/

 This report confirms that all midterm project requirements have been fully met.

Al-Razi University · Big Data – Practical · Fall 2026

 August 29, 2026